Team project: Build it together

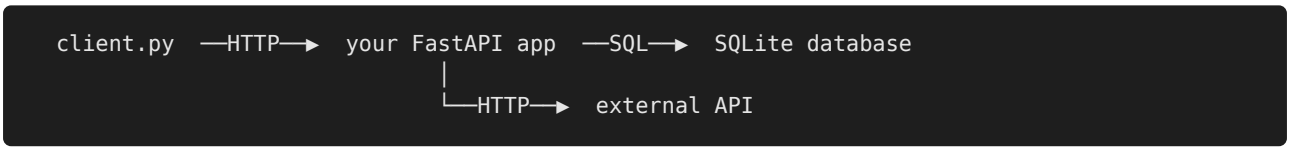
For two weeks your group works as a **Scrum Team**. Together you build a **REST API** with FastAPI and a **SQLite database**, connect it to an **external API**, and write a **Python client** that uses it. Everything from the course so far comes together here: Python, databases, APIs, Git and Scrum.

There are **no lectures** in this module. Your group plans, builds, reviews and presents. Your group makes its own decisions.

The project at a glance

What	Details
Period	26 October – 6 November 2026
Team	Your group, working as one Scrum Team
Sprints	Two sprints, one week each
Where	Your group channel in Teams
Product	A FastAPI + SQLite API that uses an external API, and a Python client that uses your API
Case	You choose one of four cases (see Choose your case)
Final presentation	Friday 6 November, max 15 minutes per group

What you build



After this project you can

- Work in a group using agile methods
- Plan, distribute and follow up work in a backlog
- Deliver a working application together with others
- Give and receive code review

How you work

You work the same hours as usual, 08:00–17:00, but the day looks different. Nobody stands at the front. Your group runs its own day, the agile way: small steps, fast feedback, and talking to each other every day.

A normal project day

Time	What you do
08:00–09:00	**Get ready on your own.** Start your setup, pull the latest `main`, read open pull requests, and decide what you will say at the Daily Scrum. You have had an hour to prepare, so at 09:00 everyone is ready to start.
09:00–09:15	**Daily Scrum** in your group channel, led by the Scrum Master.
09:15–12:00	Sprint work: your own issues, pair programming and code reviews.
12:00–13:00	Lunch
13:00–16:45	Sprint work. Review open pull requests **before** you start something new.
16:45–17:00	**Wrap up.** Push your branch and move your cards, so the board shows the truth.

Attendance is registered as usual. **Fridays** follow the normal Friday schedule. The afternoon session is used for Sprint Review 1 and Retrospective 1 (30 October) and for the final presentations (6 November).

The Daily Scrum at 09:00

15 minutes, every day, same place. Each person answers three questions:

- What did I finish yesterday?
- What will I do today?
- Is anything blocking me?

Long discussions happen **after** the meeting, with the people who need them. The Daily Scrum is for the team, not a status report to the teacher. The Scrum Master writes a short note in `docs/daily.md`: the date and one line per person.

Roles

Role	Who	In this project
Product Owner	One person	Owns the Product Backlog and its order. Decides what is most valuable to build first. Also writes code.
Scrum Master	One person	Leads the Daily Scrum, the Review and the Retrospective. Removes blockers and asks the teacher for help when the group is stuck. Also writes code.
Developers	Everyone	Decide together how to build things. Write code and review each other's code.

Your group owns its decisions. When the case does not say something, decide as a team and write the decision down. You may switch roles between sprint 1 and sprint 2. Write down who had which role.

LEXICON

Words you will use every day

These are the agile words from the Scrum lesson, and what they mean in **this** project.

Keyword	In this project
Product Goal	One sentence: what your product is for, and for whom.
Product Backlog	All user stories, as issues in GitHub, in the order the Product Owner decides.
User story	As a <user>, I want <something>, so that <why>. Small enough to finish in one sprint.
Acceptance criteria	2–3 checks for one story: is it doing the right thing?
Story points	Size compared to other stories: 1, 2, 3, 5, 8, 13. Estimate with planning poker. Split anything bigger than 8.
Sprint Planning	At the start of each sprint: choose the stories and write a Sprint Goal.
Sprint Goal	Why this sprint matters. It does not change during the sprint.
Sprint Backlog	The stories you chose for the sprint, on your board.
Board and WIP limit	Columns To do, In progress, Review, Done. Max **one** card in progress per person.
Daily Scrum	09:00 every day, 15 minutes.
Definition of Done	One checklist for all work. A card is not Done until every point is true.
Pull request	Every change reaches `main` through a pull request.
Code review	Someone else reads your code before it is merged, and you read theirs.
Pair programming	Two people, one shared screen. One types, one thinks ahead. Swap often.
Increment	The working product at the end of a sprint.
Sprint Review	Show the increment and get feedback.
Retrospective	How can we work better? Agree on one change and try it in the next sprint.

Start of your Definition of Done

Your group writes its own Definition of Done on day 1. Start from this list and add your own points:

- The acceptance criteria are met
- Tested by hand in `/docs`, and in `client.py` if the client uses it
- Follows our coding standard in `CONTRIBUTING.md`
- Reviewed and approved by at least one other person
- Merged to `main`, and `main` still runs
- No keys or passwords in the code

LEXICON

Timeline

Day	Date	What happens
Mon	26 Oct	**Planning day.** Choose a case, set up the team and the repository, and run Sprint Planning 1 (see below).
Tue–Thu	27–29 Oct	**Sprint 1.** Daily Scrum at 09:00. Build, review, merge.
Fri	30 Oct	Afternoon: **Sprint Review 1** (a 5-minute demo for the class) and **Retrospective 1**.
Mon	2 Nov	Backlog refinement and **Sprint Planning 2**.
Tue–Wed	3–4 Nov	**Sprint 2.** Finish the product. New ideas go into the backlog, not into the running sprint.
Thu	5 Nov	**Feature freeze at 12:00.** After that: fix bugs, finish the README, Retrospective 2 and rehearse the presentation. **Code is handed in at 17:00.**
Fri	6 Nov	Afternoon: **final presentations.**

Planning day, step by step

By the end of Monday you have a repository everyone can clone, a board with ordered stories, a Definition of Done, a coding standard, an ER diagram and a Sprint Goal.

1 Choose a case
Read the four cases and choose one. Tell your teacher before 10:30.

2 Choose roles and a Product Goal
Choose a Product Owner and a Scrum Master. Write the Product Goal in one sentence.

3 Write the Product Backlog
10–15 user stories as GitHub issues. Add acceptance criteria to the top five. The Product Owner puts them in order, most valuable first.

4 Agree on how you work
Write your Definition of Done and your coding standard in `CONTRIBUTING.md`: naming, PEP 8, docstrings, branch names and how you review.

5 Set up the repository and the board
One person creates the repository, invites the others and the teacher, and adds the folder structure, `.gitignore`, `.env.example` and `requirements.txt`. This first setup is the **only** push straight to `main`. Create the board in GitHub Projects.

6 Draw the ER diagram
Together, before anybody writes a `CREATE TABLE`.

7 Sprint Planning 1
Estimate the top stories with planning poker, choose what fits in the sprint, and write a Sprint Goal. Tip: a good first goal is the database and the basic CRUD routes work.

Choose your case

Choose **one** of four cases. All cases have the same technical requirements. Each one has a customer, users, data with relationships, personal data and an external API. Something not covered? Make a sensible decision as a team and write it down.

Case 1: FitLife Booking

> You planned the FitLife app in the Scrum exercise. Now you build it. FitLife has grown: it has gyms in three cities and has started outdoor classes in the park. Members want to see the weather before they book an outdoor class.

Data: members, instructors, gyms, classes (type, time, max places, indoor or outdoor) and bookings. A member books many classes, and a class has many members.

External API: Open-Meteo (open, no key). The gym's city gives coordinates, and the coordinates give a forecast.

```
# City -> coordinates
https://geocoding-api.open-meteo.com/v1/search
    params: name=Uppsala, count=1
# Coordinates -> forecast for the coming days
https://api.open-meteo.com/v1/forecast
    params: latitude, longitude, hourly=temperature_2m,precipitation_probability
```

Story ideas

- Book a class, and get a clear error if it is full
- Cancel a booking
- An instructor sees who has booked a class
- The most popular class types (statistics)
- Members who have never booked a class
- The weather forecast for an outdoor class

Personal data: names, emails and phone numbers of members, and who booked what. Do you really need a birth date?

Case 2: Book Club Library

> We are a small community library run by volunteers. Today we write loans in a notebook. We want a system for our books, members and loans. Adding a new book should not mean typing everything by hand.

Data: books, authors, members and loans (loan date, due date, return date). A book can have many authors, and an author can write many books.

External API: Open Library (open, no key). Search for a book and use the title, the authors and the year when you add it.

```
https://openlibrary.org/search.json
    params: title=the hobbit, limit=5
    # the answer has a key docs: a list with title, author_name, first_publish_year
```

Story ideas

- Add a book with data from Open Library
- Borrow and return a book, and get a clear error if it is already out
- Loans that are overdue
- The most borrowed books (statistics)
- Books that have never been borrowed

Personal data: names, emails and phone numbers of members, and what each person has borrowed. A reading history can say a lot about a person.

Case 3: Dev Team Portfolio

> We are a small IT consultancy with about 30 developers. When a customer asks for someone who knows Python and SQL, we search through old emails. We want one place for our developers, their skills and our projects, and we want to see what each developer has been building on GitHub.

Data: developers, skills, projects and assignments. A developer has many skills (with a level from 1 to 5), and a developer can work on many projects with a role.

External API: GitHub API, with a token in `.env`, like in Lab 3.

```
GET https://api.github.com/users/<username>/repos
    params: sort=pushed, per_page=5
    header: Authorization: Bearer <token>
```

Story ideas

- Find all developers with a certain skill
- Add a developer to a project with a role
- Developers who are not on any project right now
- How many developers have each skill (statistics)
- A developer's latest repositories from GitHub

Personal data: names, emails and GitHub usernames, and who works where. The consultancy also wants salary and birth date "just in case". What do you answer?

Case 4: ReLoop Second-hand Shop

> ReLoop sells second-hand clothes and electronics online. We want an API for products, customers and orders, and we want to import test products from a supplier's API instead of typing them in.

Data: categories, products (with stock), customers, orders and order items (quantity and the price paid). An order has many products, and a product is in many orders.

External API: Fake Store API (open, no key). Import products and categories into your own database.

```
GET https://fakestoreapi.com/products
GET https://fakestoreapi.com/products/categories
```

Story ideas

- Import products, without duplicates if you import twice
- Place an order: stock goes down, and you cannot buy more than is in stock
- The receipt for an order
- Revenue per category (statistics)
- Products that have never been sold

Personal data: names, emails, addresses and order history. Orders must be kept for 7 years (bokföringslagen). This case is close to the webshop from the SQL weeks: make your own design, do not copy webshop.db.

Technical requirements

Everything below is a **must**. It is the same for all four cases. Ideas for going further are under Signs of a higher level at the end.

1 Database

- An ER diagram in the README. Draw it any way you like.
- At least 4 tables in 3NF, with at least one one-to-many and one many-to-many relationship (a middle table).
- `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, `CHECK` and `DEFAULT` where they make sense.
- `schema.sql` creates the tables and `seed.py` fills them with test data, so anyone can build the database from scratch. The `.db` file is not in Git.
- `PRAGMA foreign_keys = ON` on every connection. Always `?` placeholders, never f-strings in SQL. `commit()` after every change.

2 Your API

- FastAPI with full CRUD for at least two resources: GET all, GET one, POST, PUT, PATCH and DELETE.
- Path parameters, and at least one filter with query parameters.
- Pydantic models for the body, and a separate model for PATCH (`exclude_unset=True`).
- The right status codes: 201 for create, 204 for delete, 404 for missing, 422 for wrong data and 401 for a missing or wrong key. When a rule in your case is broken, raise an `HTTPException` with a clear message.
- The data lives in SQLite, not in a Python list.

3 Questions your API can answer

- At least one endpoint that JOINs three or more tables, for example a receipt or a booking list.
- At least one statistics endpoint with `GROUP BY` and `COUNT`, `SUM` or `AVG`.
- At least one endpoint with `LEFT JOIN` and `IS NULL` that finds what is missing.

4 External API

- Called with `requests` from its own module, for example `external_api.py`, with a helper like `get_json`.
- A timeout on every request.
- If the external API is slow, down or answers with an error, your API does not crash. It answers with a clear message.
- The data is **used** in your product: saved in your database or combined with your own data, not just printed.

5 Security and personal data

- Every route that changes data needs an `X-API-Key` header. The key is in `.env`, `.env` is in `.gitignore`, and `.env.example` (without the real key) is in Git. If the key is missing in `.env`, the API refuses everybody.
- A section in the README: which columns are personal data, why you need each one, and what you chose not to store.
- An endpoint that shows everything you store about one person: What do you have about me?
- An endpoint for Delete me. If you must keep the history, anonymise the person instead, and explain why in the README.

6 Python client and code quality

- `client.py`: a menu program that uses your API with `requests`. It can list, show one, create, update and delete, and show at least one statistics endpoint. It reads the key from `.env`.
- The code is split into modules with clear names. Functions do one thing. `try/except` where things can go wrong.
- At least one class of your own (not counting Pydantic models) where it makes the code clearer, for example a class that wraps the external API.
- PEP 8 and your own coding standard. No dead code or commented-out experiments in `main`.
- `requirements.txt` and a README that explains how to install, seed and run everything.

A structure you can start from

```
project/
    main.py             # the FastAPI app and the routes
    database.py         # connection and SQL queries
    models.py           # Pydantic models
    external_api.py     # get_json and the calls to the external API
    client.py           # the menu program that uses your API
    schema.sql          # CREATE TABLE statements
    seed.py             # builds the database and adds test data
    requirements.txt
    .env.example        # API_KEY=change-me
    .gitignore          # .env, *.db, __pycache__/, .venv/
    README.md
    CONTRIBUTING.md
    docs/               # daily.md, retro-sprint-1.md, retro-sprint-2.md, ER diagram
```

Good to know

Tip	Why
One connection per call	Open a new connection in each database function (connect, execute, commit, close). Do not share one global connection between requests.
Reset the database	Delete the `.db` file and run `seed.py` again. Everyone gets the same test data.
Rows as dictionaries	`conn.row_factory = sqlite3.Row` lets you write `dict(row)`, which is handy when you return JSON.
Two terminals	One runs `fastapi dev main.py`, the other runs `client.py`.

Git and teamwork rules

Your teacher reads your commit history, pull requests, reviews and board. That is where your own contribution is seen, so make your work visible there.

1. **One repository per group**
   On GitHub. Everyone in the group is a collaborator, and the teacher is invited. The board is a GitHub Project linked to the repository.

2. **main always works**
   Nobody pushes straight to `main`, except the very first setup. If your repository allows it, add a rule for `main` that requires a pull request. Either way, the rule applies.

3. **One issue, one branch**
   Every story or task is an issue. Make a branch for it, named after the issue, for example `feature/12-book-a-class`. Pull `main` often so your branch does not fall behind.

4. **Every change through a pull request**
   Keep pull requests small. Link the issue with `Closes #12`. Write what you changed and how to test it.

5. **Review before merge**
   At least one other person approves before the merge. A good review says what works well and asks a question or suggests an improvement. "LGTM" alone is not a review. The author fixes the comments and merges.

6. **Merge conflicts**
   Solve them, do not avoid them. If a conflict touches someone else's code, solve it together in a call.

7. **Everyone builds**
   Each person authors **at least 3** merged pull requests and reviews **at least 3**. Spread the work so that everyone works on both the database and the API side.

8. **Keep a record**
   `docs/daily.md` (one note per Daily Scrum), `docs/retro-sprint-1.md`, `docs/retro-sprint-2.md` and `CONTRIBUTING.md`.

When you are stuck

Try in this order, and move on to the next step only when you are still stuck:

- **1. Yourself.** Read the error message from the bottom up, print the data you are working with, and test in `/docs`.
- **2. Your group.** Ask at the Daily Scrum or in the channel. Pair program on the problem.
- **3. The internet.** Read the documentation (FastAPI, requests, SQLite, the external API) and search for the error message.
- **4. The teacher.** The Scrum Master asks the teacher, and says what the group has already tried.

LEXICON

Final presentation and hand-in

Friday 6 November. Each group presents on its own, **max 15 minutes**. Everyone in the group presents a part.

Part	Time	What you show
1. The product	about 5 min	Show the **finished** product first. What can a user do? Show it live in `/docs` and `client.py`. Tell it the way a user would use it.
2. How it is built	about 4 min	The ER diagram and how the parts fit together. Choose one or two pieces of code you are proud of, or learned the most from, and explain the idea behind them. Do not go through the code line by line.
3. How you worked	about 4 min	Your board, a pull request where a review changed something, a problem and how you solved it, and what your retrospectives changed.
4. Questions	about 2 min	

Tips

- Rehearse on Thursday afternoon and time it.
- Decide in advance who shares the screen and who says what.
- Start the server and seed the database **before** it is your turn.
- Have a plan B, like screenshots, in case something breaks.

Hand-in

The group hands in **one** joint delivery, the usual way, by **Thursday 5 November, 17:00**: the link to your repository. What is on `main` at the deadline is what we look at. The repository contains:

- All the code, `schema.sql`, `seed.py` and `requirements.txt`
- The README: how to run, the endpoints, the ER diagram and the personal data section
- The `docs/` folder: daily notes, both retrospectives and the team reflection

Team reflection

Write it together in `docs/reflection.md`, for example right after Retrospective 2:

- Who had which role, and what each person mainly built
- One review that changed something, and what changed
- One problem that was hard, and how you solved it
- What worked well in your Scrum process, and one thing you would do differently next time

How your work is assessed

There are no grades in this course. The group gets **written feedback** on the whole project. Everyone's contribution and collaboration are also part of the assessment, and they are seen in GitHub and in the presentation, so you do not hand in anything on your own.

What	What we look at
Joint delivery	All the must-haves are there. Someone outside the group can clone the repository, follow the README and run everything.
Contribution	Commits, pull requests and reviews, cards on the board, and each person's part of the presentation.
Collaboration	Taking part in the Daily Scrums, reviews and retrospectives. Helping others. Making your work visible.

Signs of a higher level

Not required, but this is what makes a solution stand out:

- Reviews that find real problems or improve the design, and pull requests that are small and well described.
- A clean history: clear commit messages, conflicts solved well, and a `main` that never breaks.
- A database design you can defend: you can explain why a relationship is 1:N or N:M, and why it is in 3NF.
- An index on a column you search often, checked with `EXPLAIN QUERY PLAN`.
- A view that hides personal data from someone who does not need it.
- A cache for the external API, like in the Lab 4 bonus.
- Pages for long lists, for example `?limit=10&page=2`.
- One part of the product that uses MongoDB, where documents fit better than tables, with a short explanation of why.
- Retrospectives that lead to real changes you can show.
- A presentation that a non-technical person can follow.